

Exp.No:15
Date:

MINI PROJECT: SECURE FILE ENCRYPTION & DECRYPTION USING AES

AIM:

To design and implement a secure file encryption and decryption system that protects the confidentiality and integrity of files using symmetric cryptography.

OBJECTIVES:

- To protect sensitive files from unauthorized access.
- To use symmetric key cryptography for file encryption and decryption.
- To ensure that an encrypted file cannot be read directly without the secret key.
- To demonstrate a practical application of cryptography in data protection.

TOOLS AND TECHNOLOGIES:

- **Programming Language:** Python 3
- **Libraries:** cryptography, os
- **Encryption Method:** Fernet symmetric encryption based on AES
- **Platform:** Windows / Linux / macOS

PROCEDURE:

1. Key Generation:

Generate a secret key using the cryptography library and store it securely.

2. File Selection:

Select the file that has to be protected.

3. Encryption:

Read the file data, encrypt it using the secret key, and save the ciphertext as an encrypted file.

4. Decryption:

Use the same secret key to decrypt the encrypted file and restore the original content.

5. Verification:

Compare the restored file with the original file to verify successful recovery.

EXPLANATION:

This project provides a simple file protection mechanism using symmetric cryptography. A secret key is used for both encryption and decryption. When a user selects a file, the system reads its binary data and encrypts it before storing the protected output.

The encrypted file contains ciphertext rather than readable original content. Only a person who possesses the correct secret key can decrypt the ciphertext and recover the original file. This demonstrates the use of cryptography for protecting sensitive data such as documents, notes, and other files.

SYSTEM FLOW:

Select File → Generate/Load Secret Key → Encrypt File → Encrypted File → Decrypt Using Same Key → Original File

IMPLEMENTATION:

Install dependencies

```
pip install cryptography
```

Server Code (secure_server.py)

```
from cryptography.fernet import Fernet
```

```
# Generate secret key
```

```
def generate_key():
```

```
    key = Fernet.generate_key()
```

```
    with open("secret.key", "wb") as file:
```

```
        file.write(key)
```

```
    return key
```

```
# Load secret key
```

```
def load_key():
```

```
    with open("secret.key", "rb") as file:
```

```
        return file.read()
```

```
# Encrypt file
```

```
def encrypt_file(filename, key):  
    cipher = Fernet(key)  
    with open(filename, "rb") as file:  
        data = file.read()  
    encrypted_data = cipher.encrypt(data)  
    with open(filename + ".enc", "wb") as file:  
        file.write(encrypted_data)  
    print("File encrypted successfully!")  
    print("Encrypted file:", filename + ".enc")
```

```
# Decrypt file
```

```
def decrypt_file(filename, key):  
    cipher = Fernet(key)  
    with open(filename, "rb") as file:  
        encrypted_data = file.read()  
    decrypted_data = cipher.decrypt(encrypted_data)  
    output_file = filename.replace(".enc", "_decrypted")  
    with open(output_file, "wb") as file:  
        file.write(decrypted_data)  
    print("File decrypted successfully!")  
    print("Decrypted file:", output_file)
```

```
# Main program
```

```
print("=====  
print(" SECURE FILE ENCRYPTION SYSTEM")  
print("=====  
print("1. Encrypt File")  
print("2. Decrypt File")
```

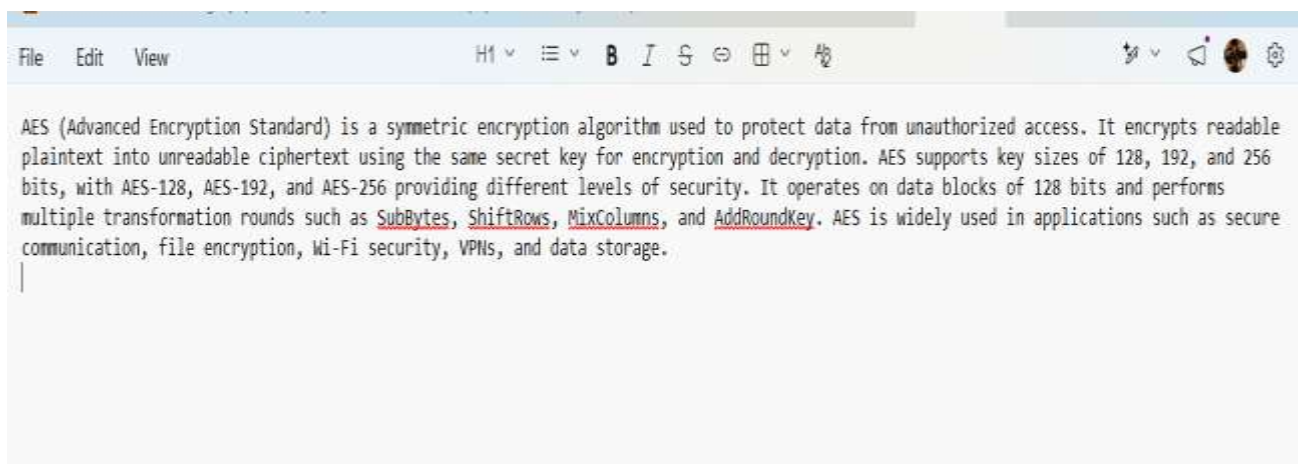
```

choice = input("Enter your choice: ")
if choice == "1":
    key = generate_key()
    filename = input("Enter file name: ")
    encrypt_file(filename, key)
    print("Keep the secret.key file safe!")
elif choice == "2":
    filename = input("Enter encrypted file name: ")
    key = load_key()
    decrypt_file(filename, key)
else:
    print("Invalid choice!")

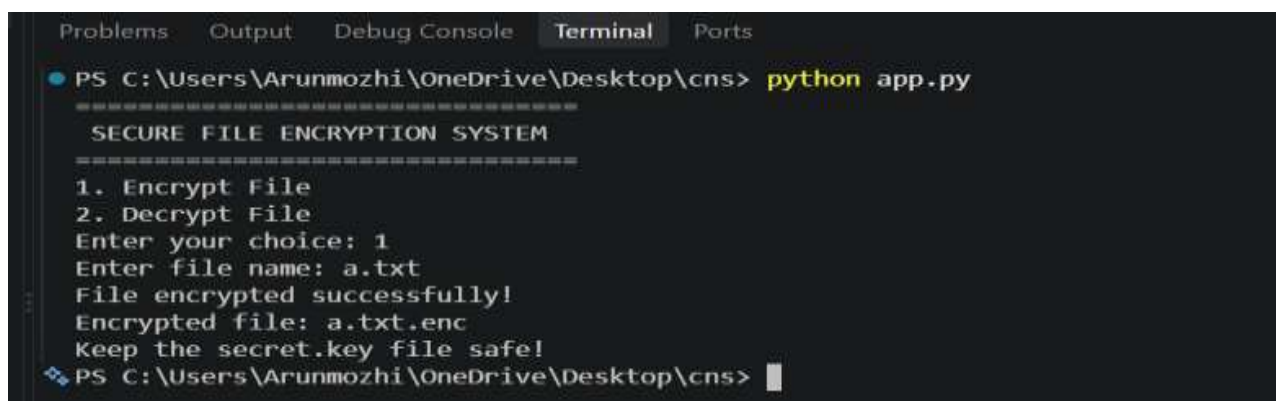
```

OUTPUT:

Normal File:



Encryption:



```
File Edit View

gAAAAABqu611FS0bv6nTHKx8EnUqJ7FgZ51xfSkRRAn6GnpIDmbKxudP_6b6SAtraTZJ7CgffW5ngj1iPMkvptEUHf4DtcEntK1grqH3s5QbqBrBqT-Z3-cRN10FjN8HKyc5Je_
5J4M4xmbJ9_3zaqncouz0ZG4j7ckmTjt1AHmLmbC10UinBsgnM5h1Ge4PBwEfPo66Za5ZMuXfD0fhB5jxJmCFUsA9zyPwWzag_SgMgoty_Bi9Dj8900ZoaJE5d4pd6-
kIccVQIYacZjVrLdewsfSsDM_ZYTUDLbpvQJ5TwNBK5QUqqkNueVvtpPhyIribBYTR76yfcCD9evlCY0o6LhKkS1VOC9B5YUtDaBzsTwXJh5FWA5WqksRBWZ4Xrkh8ZikoSoRs901A
KAeLmwnoiehwzrvEg9q2208lg4FLex%ad0vN-fZii_k8EZJw0NF1p-3YHN9
_rgsJ3FTb22rzlae1STGw83bQwmp7a9sp4Rn1Bj1Rum3DaYNar8rKFr1dPFx8AYMGLAlt1kVMBEJMb-IPg5jw_EINSz6ns6uM0-
Yj8Uer3Vn3acET08Tn8ePMb1xSd_7z83r6lQQeykS1yH3TlVKaeHfxkm92Efevlpm7uUMTJbo35QZvgc4CLfAXbN1eIBET1qkWH53eY9VVQ_Kg56m0wSwgM4n8EpeL-
jnbUyqle5zyN0VY1jTkZ1Mc4B2VL3pxnW6247wPBHCg0iYSYrtA6QhESTQ7eCk36cMM96qnMVYKldq56-9vAfe9Q8J9PM8h-w_gDvNMWv4JE8RQqrXZhqAFIrKACNp8j-
zhzyqK4fhrAMjbaT-waCS29ynWleELGrmzP2QuiQnMc4ZHT08UisfXJmRCfIWU4RVujGFTFPFZJDKSGKD
```

Decryption:

```
Problems Output Debug Console Terminal Ports

● PS C:\Users\Arunmozhi\OneDrive\Desktop\cns> python app.py
=====
SECURE FILE ENCRYPTION SYSTEM
=====
1. Encrypt File
2. Decrypt File
Enter your choice: 2
Enter encrypted file name: a.txt.enc
File decrypted successfully!
Decrypted file: a.txt_decrypted
❖ PS C:\Users\Arunmozhi\OneDrive\Desktop\cns>
```

```
File Edit View

AES (Advanced Encryption Standard) is a symmetric encryption algorithm used to protect data from unauthorized access. It encrypts readable
plaintext into unreadable ciphertext using the same secret key for encryption and decryption. AES supports key sizes of 128, 192, and 256
bits, with AES-128, AES-192, and AES-256 providing different levels of security. It operates on data blocks of 128 bits and performs
multiple transformation rounds such as SubBytes, ShiftRows, MixColumns, and AddRoundKey. AES is widely used in applications such as secure
communication, file encryption, Wi-Fi security, VPNs, and data storage.
```

RESULT:

The system successfully enables secure, encrypted text communication between client and server. Messages intercepted over the network appear as unreadable ciphertext, ensuring confidentiality.